Quantum-Safe File Encryptor (QSFE)

A Standards-Aligned Hybrid Cryptographic Proof of Concept for Post-Quantum Data Protection

---

⚠️ Important Security Warning

This project is an **educational Proof of Concept (PoC)**.

- It has **not been formally audited** by cryptographers.
- It is **not intended for production use**.
- The implementation uses a **static ML-KEM keypair (no forward secrecy)**.
- Several security hardening measures (e.g., memory zeroization, side-channel resistance) are **not implemented**.

👉 **Do NOT use this tool to protect sensitive or production data.**

Executive Summary

Quantum computing introduces a fundamental disruption to modern cryptography. Widely deployed systems such as RSA and ECC are vulnerable to quantum attacks, particularly via Shor's algorithm.

The **Quantum Safe File Encryptor (QSFE)** is a **working reference implementation** demonstrating a correct hybrid post-quantum encryption architecture, combining:

- **ML-KEM-768 (NIST FIPS 203)** for quantum-safe key encapsulation
- **HKDF-SHA256 (RFC 5869)** for secure key derivation
- **AES-256-GCM (NIST SP 800-38D)** for authenticated encryption

The system implements a **best-practice hybrid pipeline**:

> **KEM → HKDF → AEAD**

This ensures:

- Resistance to both classical and quantum attacks
- Protection against "Harvest Now, Decrypt Later" (HNDL) threats
- Strong confidentiality and integrity guarantees

QSFE is designed to help **developers, engineers, and security practitioners** understand and experiment with real-world post-quantum cryptographic systems.

Problem Statement

The Quantum Threat

Modern cryptographic infrastructure relies on:

- RSA (integer factorization)
- ECC (elliptic curve discrete logarithms)

Both are broken by Shor's algorithm on sufficiently powerful quantum computers.

Key Risks

1. Harvest Now, Decrypt Later (HNDL)

Attackers collect encrypted data today for future decryption.

2. Long Data Lifecycles

Sensitive data must remain secure for 10–30+ years.

3. Data-at-Rest Exposure

Files, backups, and archives remain largely unprotected.

4. Migration Complexity

Cryptographic transitions are slow and operationally expensive.

Industry Context

The cybersecurity community has converged on **hybrid cryptography** as the near-term solution:

- PQC → key establishment
- Symmetric encryption → data protection

Standardization

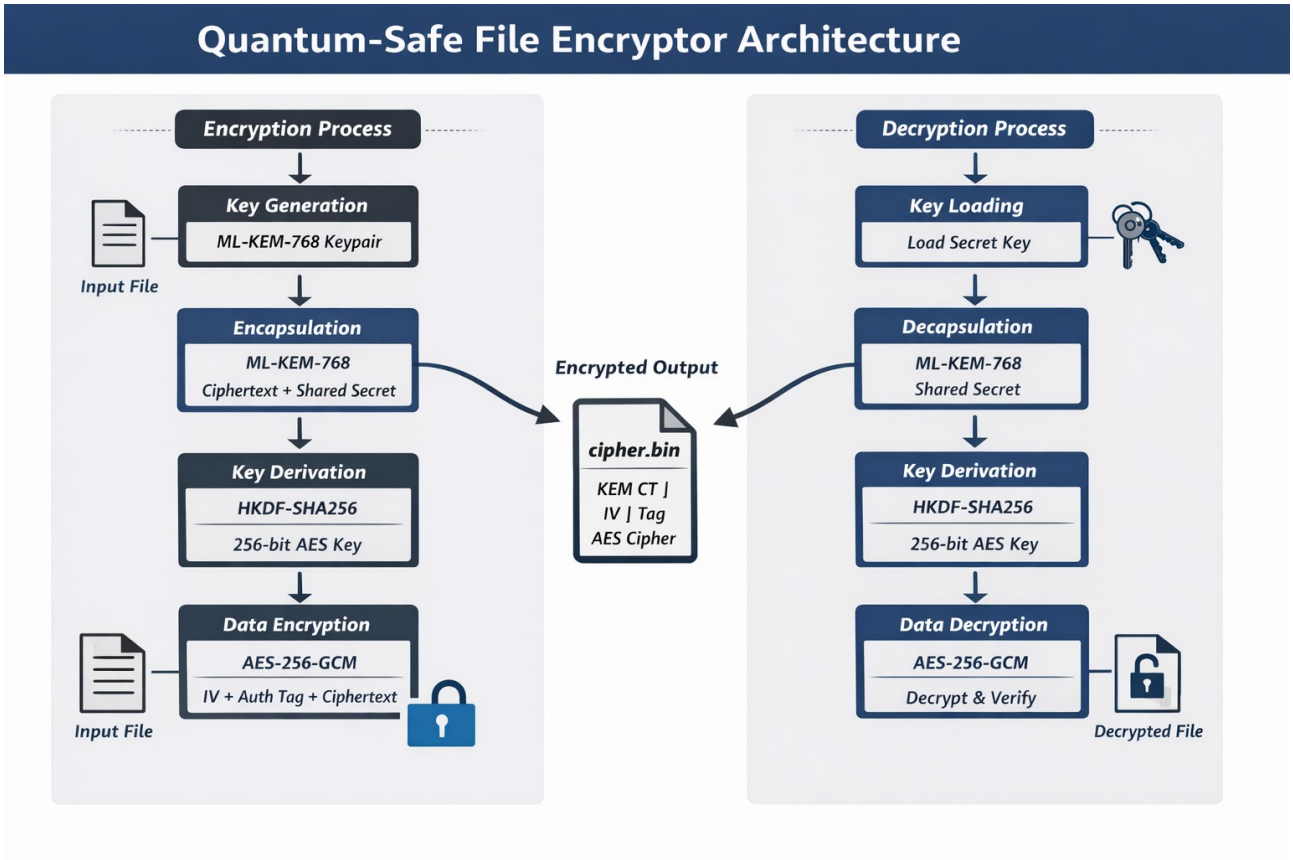
- **ML-KEM (Kyber)** standardized by NIST (FIPS 203)
- Global migration timelines: **2030–2035**

Hybrid systems like QSFE represent the **transition architecture** for the next decade.

Proposed Solution – QSFE Architecture

QSFE implements a **three-layer hybrid cryptographic pipeline**.

🔷 Architecture Diagram



Threat Model & Assumptions

Attacker Capabilities

- Can capture encrypted files and metadata
- Can perform offline cryptanalysis
- May gain future quantum capabilities

Not Covered

- Side-channel attacks
- Memory extraction attacks
- Compromised endpoints

Assumptions

- Private key remains secure
- Public key authenticity is trusted

Detailed Design

Key Generation

- Generates a **static ML-KEM keypair**
- Stored locally:
  - `mlkem_public.key`
  - `mlkem_private.key`

⚠️ No forward secrecy due to static key usage.

Encryption Flow

1. Load public key
2. Perform ML-KEM encapsulation → `(ct, ss)`
3. Derive AES key using HKDF-SHA256
4. Generate random 12-byte IV
5. Encrypt data using AES-256-GCM
6. Output structured file

Decryption Flow

1. Load private key
2. Extract KEM ciphertext
3. Perform decapsulation → shared secret
4. Derive AES key via HKDF
5. Decrypt using AES-GCM
6. Verify authentication tag

File Format Specification (Corrected & Clarified)

```
cipher.bin =
[ ML-KEM ciphertext (1088 bytes) ]
[ IV (12 bytes) ]
[ Authentication Tag (16 bytes) ]
[ AES ciphertext (variable length) ]
```

🔧 **Correction applied:** Tag comes before AES ciphertext (aligned with implementation).

Code Organisation

- `mlkem_helper.c`
  - ML-KEM operations (liboqs)
  - HKDF-SHA256
  - AES-256-GCM
- `quantum_safe_file_cli.adb`

- CLI logic
- File handling
- Workflow orchestration

This separation ensures:

- Performance (C layer)
- Portability (Ada layer)

Build & Installation

Requirements

•GNAT Ada compiler

•liboqs (Open Quantum Safe library)

•OpenSSL (libcrypto)

Step-by-Step (Ubuntu/Debian example)

bash

```
# 1. Install dependencies
sudo apt update
sudo apt install gnat cmake ninja-build build-essential libssl-dev
# 2. Build and install liboqs
git clone https://github.com/open-quantum-safe/liboqs.git
cd liboqs
mkdir build && cd build
cmake -GNinja -DCMAKE_INSTALL_PREFIX=/usr/local -DBUILD_SHARED_LIBS=ON ..
ninja
sudo ninja install
sudo ldconfig
# 3. Compile the C helper
cd ../..
# back to project root
gcc -O2 -o mlkem_helper mlkem_helper.c -loqs -lcrypto
# 4. Compile the Ada program
gnatmake -O2 quantum_safe_file_cli.adb
```

Limitations (Enhanced – Important Upgrade)

Critical Limitations

- **Static keypair (no forward secrecy)**
- **No key rotation mechanism**
- **Entire file loaded into memory (no streaming)**
- **No metadata authentication (KEM CT + IV not authenticated)**
- **No memory zeroization of secrets**
- **No cryptographic audit**

⚠️ Important Security Note

While AES-GCM authenticates the ciphertext, the current design does **not authenticate the ML-KEM ciphertext or IV**, which could enable certain tampering or substitution attacks in advanced threat models.

Future Improvements

To evolve QSFE toward production readiness:

- Ephemeral keys → forward secrecy
- Metadata authentication (AAD or structured headers)
- Streaming encryption for large files
- Secure memory handling (zeroization)
- Algorithm agility (support multiple PQC schemes)
- Integration with KMS / HSM

References

•NIST FIPS 203 (ML-KEM): https://csrc.nist.gov/pubs/fips/203/final

•Open Quantum Safe (liboqs): https://github.com/open-quantum-safe/liboqs

•HKDF RFC 5869: https://datatracker.ietf.org/doc/html/rfc5869

•AES-GCM: NIST SP 800-38D